

Bloc 2 TP Symfony

Contexte	2
Initialisation & Configuration	2
Modélisation des Données	4
Back-End & Génération du CRUD	5
Integration Front-End & Twig	7
Intégration en autonomie	9
Conclusion	11

Contexte

Ce TP consiste à créer une application web de gestion d'événements avec **Symfony** & **MySQL** & **Twig** (Le projet sera disponible sur **GitHub** (https://github.com/RomAub/bvl_S3_B2_TP_Symfony/tree/main)).

Initialisation & Configuration

Création du projet **Symfony** avec la commande :

```
symfony new eventify --webapp
```

Puis on rentre dans le dossier créé avec :

```
cd eventify
```

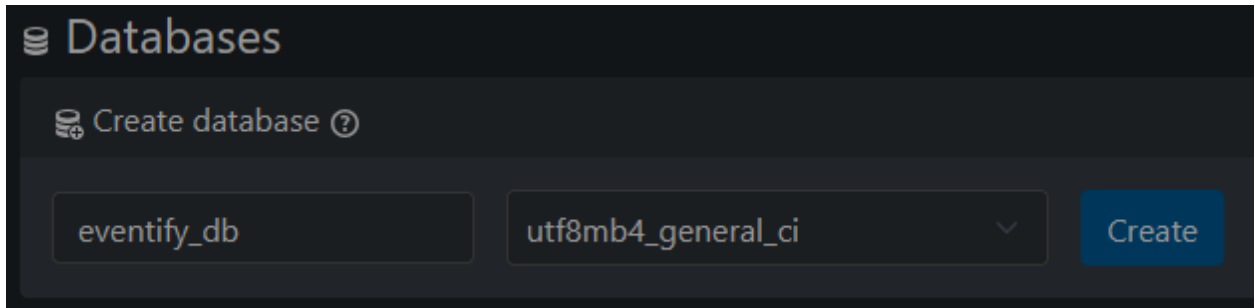
Résultat :

```
C:\xampp\htdocs>set HTTP_PROXY=http://172.16.63.130:3128
C:\xampp\htdocs>set HTTPS_PROXY=http://172.16.63.130:3128
C:\xampp\htdocs>rmdir /s /q eventify
C:\xampp\htdocs>symfony new eventify --webapp
* Creating a new Symfony project with Composer
* Setting up the project under Git version control
  (running git init C:\xampp\htdocs\eventify)

[OK] Your project is now ready in C:\xampp\htdocs\eventify

C:\xampp\htdocs>cd eventify
C:\xampp\htdocs\eventify>_
```

Après avoir fait cela on doit créer la **BDD** (sur **phpMyAdmin**) :



The screenshot shows the 'Databases' section of phpMyAdmin. At the top, there's a 'Create database' link with a question mark icon. Below it, there's a form with two input fields: the first contains 'eventify_db' and the second contains 'utf8mb4_general_ci' with a dropdown arrow. To the right of these fields is a blue 'Create' button.

Puis dans le fichier “**.env**” dans le dossier du projet eventify on modifie “**DATABASE_URL=**” pour contenir :

```
DATABASE_URL="mysql://root:@127.0.0.1:3306/eventify_db?serverVersion=8.0  
&charset=utf8mb4"
```

Ensuite on vérifie la connexion en créant la base directement depuis **Symfony** :

```
C:\xampp\htdocs\eventify>php bin/console doctrine:database:create  
Could not create database 'eventify_db' for connection named default  
An exception occurred while executing a query: SQLSTATE[HY000]: General error: 1007  
Can't create database 'eventify_db'; database exists
```

Le message confirme que la **BDD** existe déjà.

Modélisation des Données

Création de l'entité “**Event**” avec :

```
php bin/console make:entity Event
```

Il faut ensuite entrer le nom, le type, la taille (si besoin) et si la valeur peut être null.

```
Add another property? Enter the property name (or press <return> to stop adding fields):
> capacity

Field type (enter ? to see all types) [string]:
> integer

Can this field be null in the database (nullable) (yes/no) [no]:
> n

updated: src/Entity/Event.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>

Success!

Next: When you're ready, create a migration with php bin/console make:migration
```

Puis on doit migrer les données avec :

```
php bin/console make:migration
```

```
php bin/console doctrine:migrations:migrate
```

```
C:\xampp\htdocs\eventify>php bin/console make:migration
created: migrations/Version20260903111949.php

Success!

Review the new migration then run it with php bin/console doctrine:migrations:migrate
See https://symfony.com/doc/current/bundles/DoctrineMigrationsBundle/index.html

C:\xampp\htdocs\eventify>php bin/console doctrine:migrations:migrate

WARNING! You are about to execute a migration in database "eventify_db" that could result in schema changes and data loss. Are you sure you wish to continue? (yes/no) [yes]:
> y

[notice] Migrating up to DoctrineMigrations\Version20260903111949
[notice] finished in 41.7ms, used 22M memory, 1 migrations executed, 2 sql queries

[OK] Successfully migrated to version: DoctrineMigrations\Version20260903111949
```

Back-End & Génération du CRUD

Pour accélérer le développement du **Back-End**, nous allons générer le contrôleur et les formulaires automatiquement avec :

```
php bin/console make:crud Event
```

```
C:\xampp\htdocs\eventify>php bin/console make:crud Event

Choose a name for your controller class (e.g. EventController) [EventController]:
> EventController

Do you want to generate PHPUnit tests? [Experimental] (yes/no) [no]:
> y

created: src/Controller/EventController.php
created: src/Form/EventType.php
created: templates/event/_delete_form.html.twig
created: templates/event/_form.html.twig
created: templates/event/edit.html.twig
created: templates/event/index.html.twig
created: templates/event/new.html.twig
created: templates/event/show.html.twig
created: tests/Controller/EventControllerTest.php

Success!

Next: Check your new CRUD by going to /event/
```

Dans “C:\xampp\htdocs\eventify\src\Entity\Event.php”, on ajoute des règles de validation grâce à **Symfony**.

```
<?php

namespace App\Entity;

use App\Repository\EventRepository;
use Doctrine\DBAL\Types\Types;
use Doctrine\ORM\Mapping as ORM;
use Symfony\Component\Validator\Constraints as Assert;

#[ORM\Entity(repositoryClass: EventRepository::class)]
class Event
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 255)]
    #[Assert\NotBlank(message: "Le titre ne peut pas être vide.")]
    #[Assert\Length(min: 5, minMessage: "Le titre doit faire au moins {{ limit }} caractères.")]
    private ?string $title = null;

    #[ORM\Column(type: Types::TEXT)]
    #[Assert\Positive(message: "La description ne peut pas être vide.")]
    private ?string $description = null;

    #[ORM\Column]
    #[Assert\NotBlank(message: "La date est obligatoire.")]
    private ?\DateTime $eventDate = null;

    #[ORM\Column(length: 255)]
    #[Assert\NotBlank(message: "Le lieu doit être renseigné.")]
    private ?string $location = null;

    #[ORM\Column]
    #[Assert\Positive(message: "La capacité doit être un nombre positif.")]
    private ?int $capacity = null;
}
```

Integration Front-End & Twig

Dans “*templates/base.html.twig*”, CDN CSS (Bootstrap 5 dans ce contexte) dans le **<head>** pour habiller l'interface. Cela permet d'avoir des rendu propre en CSS sans avoir à les créer, c'est déjà tout fait.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>{% block title %}Eventify{% endblock %}</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  </head>
  <body class="bg-light">
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
      <div class="container">
        <a class="navbar-link navbar-brand fw-bold" href="{{ path('app_event_index') }}">🎉 Eventify</a>
        <a href="{{ path('app_event_new') }}" class="btn btn-success">+ Créer un événement</a>
      </div>
    </nav>

    <div class="container">
      {% block body %}{% endblock %}
    </div>
  </body>
</html>
```

Reformatage de la liste dans “*templates/base.html.twig/event/index.html.twig*”, Cela permet d'afficher les événements sous forme de cartes plutôt que dans un tableau HTML :

```
<div class="container">
  {% extends 'base.html.twig' %}

  {% block title %}Liste des événements{% endblock %}

  {% block body %}
    <h1 class="mb-4">Événements à venir</h1>

    <div class="row">
      {% for event in events %}
        <div class="col-md-4 mb-4">
          <div class="card h-100 shadow-sm">
            <div class="card-body">
              <h5 class="card-title">{{ event.title }}</h5>
              <p class="card-text text-muted">{{ event.description|slice(0, 100) }}...</p>
              <p class="mb-1"><strong>📅 Date :</strong> {{ event.eventDate|date('d/m/Y H:i') }}</p>
              <p class="mb-1"><strong>📍 Lieu :</strong> {{ event.location }}</p>
              <p class="mb-3"><strong>👤 Places :</strong> {{ event.capacity }}</p>

              <a href="{{ path('app_event_show', {'id': event.id}) }}" class="btn btn-outline-primary btn-sm">Voir plus</a>
              <a href="{{ path('app_event_edit', {'id': event.id}) }}" class="btn btn-outline-secondary btn-sm">Modifier</a>
            </div>
          </div>
        </div>
      {% else %}
        <p class="text-muted">Aucun événement trouvé.</p>
      {% endfor %}
    </div>
  {% endblock %}
</div>
```

Intégration en autonomie

Recherche / Filtre (Back-End & Front-End) :

Dans “\eventify\src\Repository\EventRepository.php”:

```
public function findByTitle(string $search): array
{
    return $this->createQueryBuilder('e')
        ->andWhere('e.title LIKE :val')
        ->setParameter('val', '%' . $search . '%')
        ->getQuery()
        ->getResult();
}
```

Cela permet de rechercher des enregistrements dans une **BDD** dont le titre contient un mot ou une phrase spécifique.

Puis dans “\eventify\src\Controller\EventController.php”:

```
#[Route(name: 'app_event_index', methods: ['GET'])]
public function index(Request $request, EventRepository $eventRepository): Response
{
    $search = $request->query->get('q');

    if ($search) {
        $events = $eventRepository->findByTitle($search);
    } else {
        $events = $eventRepository->findAll();
    }

    return $this->render('event/index.html.twig', [
        'events' => $events,
    ]);
}
```

Il va vérifier ce qui se trouve dans “q” dans l’URL comme “x.fr?q=test” donc “q” sera “test”, si un texte est trouvé, il effectuera la recherche avec la fonction “*findByTitle*”, sinon, il affichera tout (*findAll*).

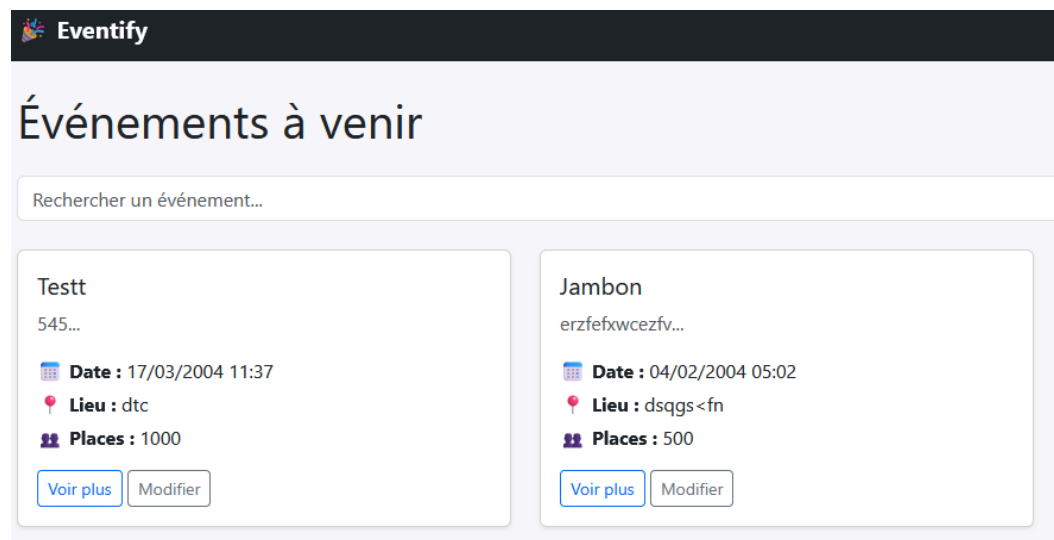
Puis dans “\eventify\templates\event\index.html.twig” :

Ajout de la barre de recherche (stockez sous “q”).

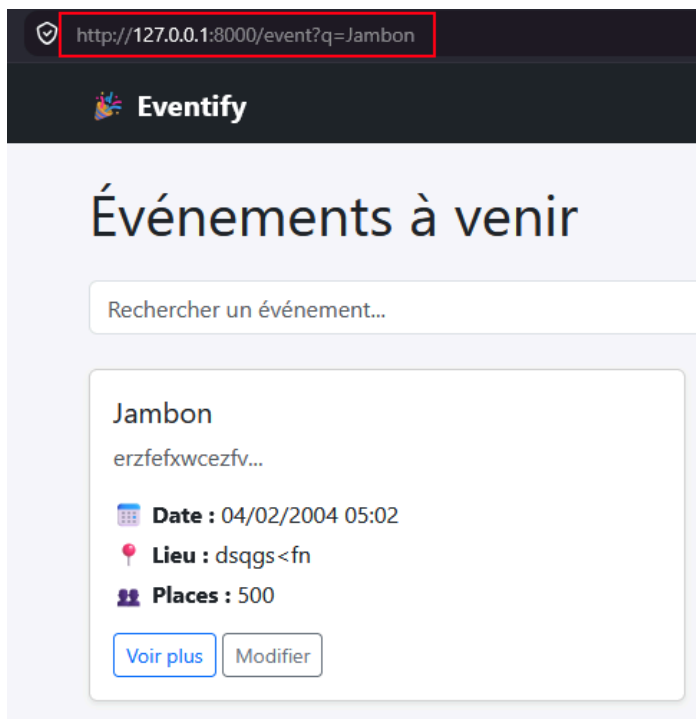
```
<form method="get" class="mb-4">
  <div class="input-group">
    <input type="text" name="q" class="form-control" placeholder="Rechercher un événement...">
    <button type="submit" class="btn btn-primary">Chercher</button>
  </div>
</form>
```

Résultat :

Page principale :



Après la recherche :



Gestion des inscriptions (Logique Métier) :

Dans “\eventify\src\Controller\EventController.php”:

```
#[Route('/{id}/register', name: 'app_event_register', methods: ['POST'])]  
public function register(Event $event, EntityManagerInterface $entityManager): Response  
{  
    if ($event->getCapacity() > 0) {  
        $event->setCapacity($event->getCapacity() - 1);  
        $entityManager->flush();  
    }  
  
    return $this->redirectToRoute('app_event_show', ['id' => $event->getId()]);  
}
```

Il gère la capacité, si il y a plus de 0 capacité disponible, il soustrait.

Puis dans “\eventify\templates\event\show.html.twig” :

```
<div class="d-flex align-items-center gap-2 mt-3">  
    <a href="{{ path('app_event_index') }}" class="btn btn-secondary">back to list</a>  
    <a href="{{ path('app_event_edit', {'id': event.id}) }}" class="btn btn-primary">edit</a>  
  
    {% if event.capacity > 0 %}  
        <form method="post" action="{{ path('app_event_register', {'id': event.id}) }}" class="m-0">  
            <button type="submit" class="btn btn-success">S'inscrire</button>  
        </form>  
    {% else %}  
        <span class="badge bg-danger p-2">Complet</span>  
    {% endif %}  
  
    {{ include('event/_delete_form.html.twig') }}  
</div>
```

Il va créer le bouton pour l’inscription, il regarde la capacité (valeur envoyée de la fonction **register**) si elle est supérieur à 0, le bouton devient cliquable, sinon, il change d’apparence et n’est plus cliquable.

Résultat :

Eventify	
Event	
Id	3
Title	fromage
Description	hethgdfg
EventDate	2026-09-26 11:29:00
Location	regerdg
Capacity	1
back to list edit	
<button>S'inscrire</button>	
Delete	

Eventify	
Event	
Id	3
Title	fromage
Description	hethgdfg
EventDate	2026-09-26 11:29:00
Location	regerdg
Capacity	0
back to list edit Complet	
Delete	

Mise en forme des Formulaires :

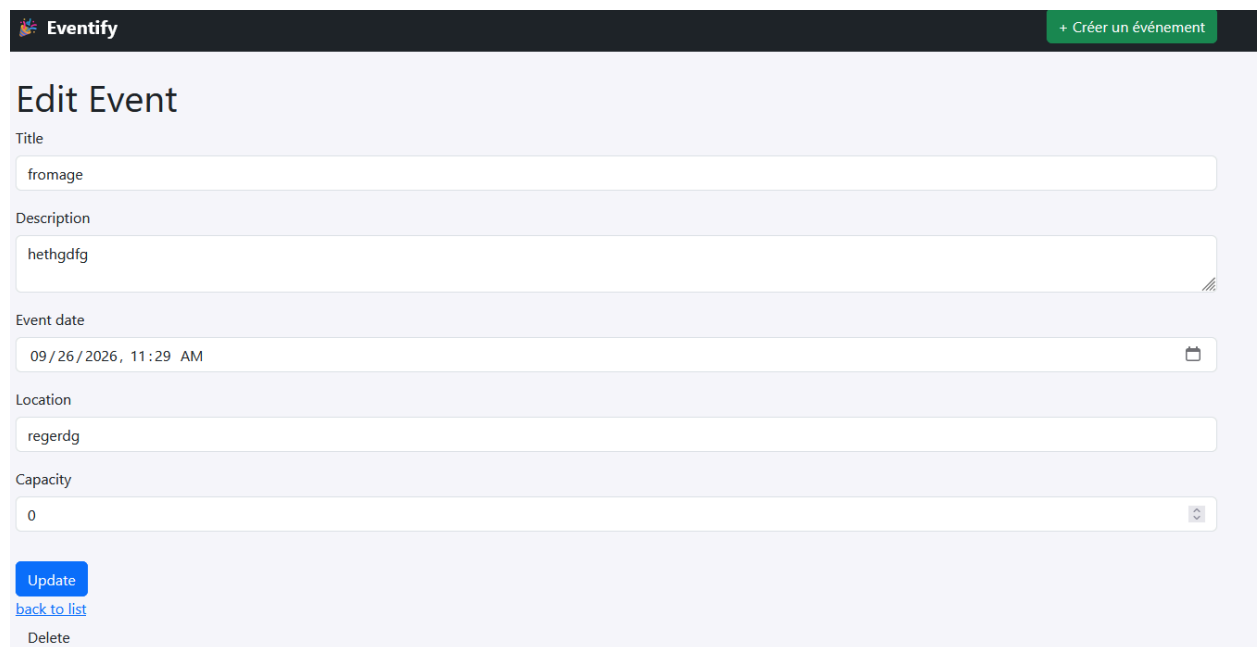
Dans "*C:\xampp\htdocs\eventify\templates\event_form.html.twig*" :

```
{% form_theme form 'bootstrap_5_layout.html.twig' %}

<span class="form-control">{{ form_start(form) }}</span>
    {{ form_widget(form) }}
    <button class='btn btn-success'>{{ button_label|default('Save') }}</button>
{{ form_end(form) }}
```

Cela va appliquer le design **Bootstrap** à toutes les pages.

Cela donnera ça :



The screenshot shows a web application interface for editing an event. At the top, there's a dark navigation bar with the 'Eventify' logo on the left and a green button labeled '+ Créer un événement' on the right. Below this, the main content area has a light purple background. The title 'Edit Event' is prominently displayed. The form consists of several input fields: 'Title' with the value 'fromage', 'Description' with 'hethgdfg', 'Event date' with '09/26/2026, 11:29 AM', 'Location' with 'regerdg', and 'Capacity' with '0'. At the bottom of the form, there are three actions: a blue 'Update' button, a blue text link 'back to list', and a grey text link 'Delete'.

Conclusion

Ce TP nous a introduit à **Symfony** et **Twig** en nous montrant à quel point il était pratique de les utiliser.